

ULTRON

Model Router & Three-Model Architecture Plan

Implementation reference for the Ultron AI coding agent • Current baseline

Purpose

This document is the architectural source of truth for how Ultron will use its three local GGUF models. It is intended to be placed in the Ultron repository so future coding agents understand the model roles, routing responsibilities, execution boundaries, and planned evolution without guessing or redesigning the architecture.

Current hardware target: Apple Silicon M4 Mac • 16 GB unified memory • llama.cpp local inference

1. Selected Model Baseline

Role	Model	Quantization	Primary responsibility
PRIMARY	Qwen3-8B	Q5_K_M	Complex reasoning, planning, general agent work, tool-use decisions
FAST	Gemma 3 4B IT	Q8_K_M	Low-complexity, latency-sensitive everyday tasks; lightweight reasoning; vision
CODING	Qwen2.5-Coder-7B-Instruct	Q8_0	Coding, debugging, repository analysis, code generation, technical repair

2. Target Ultron Architecture



Architectural rule: the Model Router selects intelligence; the Security Boundary authorizes actions. A stronger model does not gain permission to perform a higher-risk action. Security decisions remain independent.

3. Model Roles

Qwen3-8B Q5_K_M — Primary Brain

Use for complex or ambiguous tasks, multi-step reasoning, planning, general ReAct execution, interpreting difficult tool observations, and escalation/repair diagnosis. It is the default general-purpose fallback when no specialist is clearly better.

Gemma 3 4B IT Q8_K_M — Fast Path

Use for simple, latency-sensitive, non-specialist tasks such as straightforward explanations, summaries, simple queries, and lightweight reasoning. Vision is a declared capability, not an assumption that every runtime configuration automatically supports it; multimodal support must be verified by the runtime.

Qwen2.5-Coder-7B-Instruct Q8_0 — Coding Specialist

Use for code generation, debugging, refactoring, repository analysis, stack traces, implementation work, and coding-focused repair. For difficult coding failures after repeated repair attempts, Ultron may escalate reasoning to Qwen3 before returning implementation work to the coder.

4. Model Router Design

The router is a deterministic decision component. It consumes task metadata and the ModelCatalog, then returns a typed RoutingDecision. It must not load a model, call llama.cpp, execute tools, run verification, or authorize security-sensitive actions.

Routing inputs

- Complexity: SIMPLE / MODERATE / COMPLEX
- Coding requirement: whether the task is explicitly code-oriented
- Context requirement: light / normal / heavy or an equivalent typed representation
- Task state: INITIAL / IN_PROGRESS / REPAIR / ESCALATION or existing project equivalents
- Abstract memory pressure: LOW / MEDIUM / HIGH
- Required capabilities, when known

Initial routing policy

Task condition	Preferred model	Reason
Simple, non-coding	Gemma 3 4B	Lowest-cost suitable path
Moderate, non-coding	Gemma or Qwen3	Context and complexity determine whether deeper reasoning is warranted
Complex, non-coding	Qwen3 8B	Best fit for deeper general reasoning
Coding task	Qwen2.5-Coder 7B	Strongest direct coding capability
Coding + escalation	Qwen3 8B	Use stronger general reasoning to diagnose deeper/architectural failures
Simple + high memory pressure	Gemma 3 4B	Prefer lower resource demand where task capability permits

Important: these are initial policies, not permanent benchmark-derived truths. The router must remain configurable/testable so actual Ultron model-in-the-loop results can refine the weights and boundaries.

5. Hard Constraints vs Soft Preferences

Hard constraints: if a task explicitly requires a capability and a registered model lacks it, that model must not be selected. If no registered model satisfies the requirement, return a clear routing error rather than silently choosing an incompatible model.

Soft preferences: among capable models, score suitability using complexity, coding fit, context suitability, task state, memory pressure, and role preference.

Determinism: identical routing inputs and catalog state must produce the same decision.

6. RoutingDecision Contract

```
RoutingDecision
selected_model
reason
confidence
fallback_model
routing_signals
```

Confidence should be qualitative (LOW / MEDIUM / HIGH) or a documented deterministic score. It must not pretend to be an ML probability. The reason should explain why the selected model won so routing behavior can be debugged and observed.

7. Dynamic Execution Model

Model selection is not necessarily a one-time decision for a complete user task. The future execution state can cause the router to be consulted again, especially after verification failures or escalation.

```
Example:
User request
↓
Router → Coder
↓
ReAct → inspect repository → edit code
↓
Verification → FAIL
↓
Repair attempt
↓
Verification → FAIL again
↓
Escalation state
↓
Router → Qwen3
↓
deeper diagnosis
↓
Router / execution path → Coder
↓
implement repair
↓
Verification → PASS
↓
DONE
```

8. M4 16 GB Memory Strategy

The three GGUF files are not intended to remain simultaneously loaded. Their combined weight footprint is too large to treat concurrent residency as the default on a 16 GB unified-memory machine. The future ModelLifecycleManager will own loading, unloading, process reuse, and hardware-aware memory behavior.

The ModelRouter must remain hardware-agnostic. It may consume an abstract memory-pressure signal, but it must not contain M4-specific memory management or llama.cpp process logic.

Initial runtime target: begin benchmarking with an 8192-token recommended context. This is a conservative starting configuration, not a statement about the models' maximum supported context.

9. ReAct + Tools + Security Boundary

The selected model participates in the ReAct loop: reason/plan → request an action → observe the result → continue. When a tool call would have an external effect, the request passes through the Security Boundary before execution.

```
ReAct model
↓
tool request
↓
Security Boundary
■■■ ALLOW → execute tool
■■■ CONFIRM → obtain user confirmation
■■■ DENY → reject action / return decision to agent
```

The model's role, intelligence level, or confidence never overrides the Security Boundary.

10. Verification and Repair

For coding and state-changing workflows, Ultron should prefer objective verification over model self-assessment. The future loop is: generate/edit → test → inspect actual result → repair or escalate → test again. A model saying 'the fix

works' is not verification.

Verification can include tests, linting, type checks, runtime checks, and other task-specific validation already supported by the project. The verification subsystem remains separate from the ModelRouter.

11. Implementation Roadmap

Phase	Component	Status / purpose
1	ModelCatalog	APPROVED — centralized typed registry for the three models
2	ModelRouter	CURRENT — deterministic routing decision layer
3	ModelLifecycleManager	NEXT — load/unload/reuse models for 16 GB unified memory
4	AgentRuntime integration	Later — consume routing + lifecycle without breaking runtime contracts
5	ReAct integration	Later — dynamic routing during execution where justified
6	Verification / repair escalation	Later — objective validation and controlled retries
7	Model-in-the-loop benchmark	Required — validate real Ultron behavior, latency, memory, tool use, coding success

12. Non-Negotiable Architecture Rules

1. ModelCatalog is the single source of truth for model IDs, files, roles, capabilities, and quantization metadata.
2. ModelRouter selects a model; it does not load, unload, start, stop, or execute models.
3. SecurityBoundary is independent of model selection and remains the authority for ALLOW / CONFIRM / DENY.
4. Do not make high risk imply 'use a stronger model and therefore allow the action.'
5. Do not use an LLM to choose which LLM to call; routing must initially be deterministic.
6. Do not hardcode absolute machine-specific model paths.
7. Do not assume all Gemma vision functionality is active without runtime verification of the multimodal configuration.
8. Do not keep all three heavyweight models resident by default on the M4 16 GB target.
9. Do not add another model until these three have been benchmarked inside the real Ultron path.
10. Do not declare a model 'best' from model-card benchmarks alone; use Ultron's real model-in-the-loop tasks.

13. Benchmark Criteria

The three models should eventually be evaluated on the actual Ultron execution path, not only isolated prompts. At minimum measure: tool-selection accuracy, structured-output validity, ReAct completion rate, coding success, verification/repair iterations, latency, peak unified-memory usage, and hallucinated or invalid tool calls.

Success criterion: the architecture is successful when Ultron can select the appropriate specialist/general model, execute safely through the Security Boundary, verify results objectively, and escalate or repair when needed—while remaining stable on the M4 16 GB hardware target.

Document status: Architecture reference • Model baseline locked • Phase 1 approved • Phase 2 implementation in progress